Ramakrishna Mission Vidyamandira

(An Autonomous College Under University of Calcutta)

Computer Science (Honors) Semester VI 2026

Paper: 6CMSMJC4 Practical

Submitted by
Class Roll Number: 340 Registration Number: B.Sc. 6th Semester Batch: 2023-27

INDEX

SI NO.	ASSIGNMNET STATEMENT	D-O-A	D-O-S	SIGNATURE
1.	Write a LEX program to identify words and count the total number of words in a string.			
2.	Write a LEX program to identify fraction numbers.			
3.	Write a LEX program to identify the part of speech of word.			
4.	Write a LEX program to identify vowels and consonants in a given string.			
5.	Write a LEX program to identify even and odd numbers.			
6.	Write a LEX program to find the longest word in a line.			
7.	Write a LEX program to recognize SQL commands such as CREATE, INSERT, SELECT, and DELETE.			
8.	Write a LEX program to identify logical operators used in the C programming language.			
7.	Write a LEX program to find the longest word in a document (file input).			
10.	Write a LEX program to copy a C program while replacing each occurrence of the keyword float with double.			
11.	Write a LeX program to count total number of tokens that converts a file (i.e. sequence of words seperated by white space) through the following rules when u encounter a word :-			

	a. If the first letter is a consonant move it to the end of the word and then add y. b. If the first letter is a vowel , just add ly to the end of the word . All non letters are copied intact to the output.			

1. **write a LEX program to identify a word and count total number of words in a string.**

Source code:

```
%{
#include <stdio.h>
int word_count = 0;
%}
%%
[a-zA-Z]+    { word_count++; }
\n           {printf("Total number of words = %d\n", word_count);word_count
= 0;}
[ \t]+       ;
.            ;
%%

int yywrap()
{
    return 1;
}
int main()
{
    printf("Enter the String: ");
    yylex();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex Count_word.l
&& gcc lex.yy.c && ./a.out
Enter the String: My Name is Sukdeb
Total number of words = 4
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex Count_word.l
&& gcc lex.yy.c && ./a.out
Enter the String: A lexical analyzer is the first phase of a compiler that
converts input text into tokens.
Total number of words = 16
```

2. **write a LEX program to identify a fraction number.**

Source code:

```
%{
#include<stdio.h>
%}
%%
[0-9]+"/"[0-9]+    {printf("Fraction Number: %s\n",yytext);}
[ \t\n]+          ;
.                 ;
%%
int yywrap(){
    return 1;
}
int main(){
    printf("Enter the Numbers: ");
    yylex();
    return 0;
}
```

Output:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex fraction.l && gcc
lex.yy.c && ./a.out
Enter the Numbers: 3,7,3/5,8/5,2,9/13,15,75
Fraction Number: 3/5
Fraction Number: 8/5
Fraction Number: 9/13
```

3. write a LEX program to identify a part of speech of a word.

Source Code:

```
%{
#include <stdio.h>
%}
%%
(the|a|an|in|on|at|for|with|about|against|between|into|through|during|before|
after|above|below|to|from|up|down|of)  {printf("%s  :  is  a  Preposition\n",
yytext);}
(and|but|or|nor|so|yet|for) {printf("%s : is a Conjunction\n", yytext);}
(is|am|are|was|were|be|being|been) {printf("%s : is a Verb\n", yytext);}
[0-9]+ {printf("%s : is a Number\n", yytext);}
[a-zA-Z]+ {printf("%s : is a Noun\n", yytext);}
[ \t\n]+ ;
%%
int yywrap()
{
    return 1;
}
int main()
{
    printf("Enter a Sentence: ");
    yylex();
    return 0;
}
```

Output:

```
sukdeb@SUKDEB:~/sukdeb/Compile  Design  practical/lex$  flex  speech.l  &&  gcc
lex.yy.c && ./a.out */
Enter a Sentence: the boy is playing in the field
the : is a Preposition
boy : is a Noun
is : is a Verb
playing : is a Noun
in : is a Preposition
the : is a Preposition
field : is a Noun
```

4. write a LEX program to identify vowel and constant in a given string.

Source Code:

```
%{
#include <stdio.h>
%}
%%
[aeiouAEIOU] {printf("%c is a vowel\n", yytext[0]);}
[a-zA-Z]   {printf("%c is a consonant\n", yytext[0]);}
[ \t\n]      ;
%%
int yywrap()
{
    return 1;
}
int main()
{
    printf("Enter a String: ");
    yylex();
    return 0;
}
```

Output:

```
   sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex vowel_con.l
&& gcc lex.yy.c && ./a.out
   Enter a String: We live in India
   W is a consonant
   e is a vowel
   l is a consonant
   i is a vowel
   v is a consonant
   e is a vowel
   i is a vowel
   n is a consonant
   I is a vowel
   n is a consonant
   d is a consonant
   i is a vowel
   a is a vowel
```

5. write a LEX program to identify even and odd numbers.

Source Code:

```
%{
#include <stdio.h>
%}
%%
[0-9]*[02468]    { printf("Even Number: %s\n", yytext); }
[0-9]*[13579]    { printf("Odd Number: %s\n", yytext); }
%%
int yywrap(){
    return 1;
}
int main(){
    printf("Enter the Number: ");
    yylex();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex even_odd.l && gcc lex.yy.c && ./a.out
Enter the Number: 23476590
Even Number: 23476590
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex even_odd.l && gcc lex.yy.c && ./a.out
Enter the Number: 345143
Odd Number: 345143
```

6. Write a LeX program to find the longest word in a line.

Source Code:

```
%{
#include<stdio.h>
int max =0;
char word[100];
%}
%%
[a-zA-Z]+   {
                if(yyleng > max)
                    max = yyleng;
                    strcpy(word, yytext);
            }
\n          {
                printf("Length of longest word: %d\n", max);
                printf("Longest word: %s\n", word);
                max = 0; // reset for next line
            }
.   ;

%%
int yywrap(){
    return 1;
}

int main(){
    printf("Enter a line: ");
    yylex();
    return 0;

}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex1$ flex q5.l && gcc lex.yy.c
&& ./a.out
Enter a line: i am a programmer
```

```
Length of longest word: 10
Longest word: programmer
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex1$ flex q5.l && gcc lex.yy.c
&& ./a.out
Enter a line: i am a student
Length of longest word: 7
Longest word: student
```

7. **Write a LeX program to recognize SQL commands such as create , insert , select , where.**

Source Code:

```
%{
#include<stdio.h>
%}

%%

"create"|"CREATE"   { printf("CREATE command detected\n"); }
"select"|"SELECT"   { printf("SELECT command detected\n"); }
"insert"|"INSERT"   { printf("INSERT command detected\n"); }
"delete"|"DELETE"   { printf("DELETE command detected\n"); }

[a-zA-Z]+   { printf("Other word: %s\n", yytext); }
.   ;

%%

int yywrap(){
    return 1;
}

int main(){
    printf("Enter a SQL command: ");
    yylex();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex q6.l
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ gcc lex.yy.c
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ ./a.out
Enter a SQL command: select name from student
SELECT command detected
Other word: name
Other word: from
```

```
Other word: student
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex q6.l && gcc lex.yy.c && ./a.out
Enter a SQL command: INSERT INTO emp VALUES
INSERT command detected
Other word: INTO
Other word: emp
Other word: VALUES
```

8. **Write a LeX program to identify logical operator used in C language.**

Source Code:

```
%{
#include<stdio.h>
%}

%%
(==|!=|<=|>=|&&|\||!|<|>|true|false)    {printf("Logical  operator:  %s\n",
yytext);}
[ \t\n]      ;
.     {printf("Other character: %s\n", yytext); }

%%

int yywrap() {
    return 1;
}

int main() {
    printf("Enter expression:\n");
    yylex();
    return 0;
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/Compile  Design  practical/lex$  flex  q7.l  &&  gcc
lex.yy.c && ./a.out

Enter expression:

a == b && true

Other character: a

Logical operator: ==

Other character: b

Logical operator: &&

Logical operator: true
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex q7.l && gcc lex.yy.c && ./a.out

Enter expression:

a!=b | c then true

Other character: a

Logical operator: !=

Other character: b
```

9. Write a LeX program to find the longest word in a document.

Source Code:

```
%{
#include<stdio.h>
#include<string.h>

int max = 0;
char word[100] = "";
FILE *yyin;
%}

%%
[a-zA-Z]+   {
                if(yyleng > max) {
                    max = yyleng;
                    strcpy(word, yytext);
                }
            }
.   ;

%%

int yywrap(){
    return 1;
}

int main(){
    yyin = fopen("fle.txt", "r");

    if(yyin == NULL){
        printf("File not found.\n");
        return 1;
    }
    yylex();
    printf("Length of longest word: %d\n", max);
    printf("Longest word: %s\n", word);

    fclose(yyin);
    return 0;
```

```
}
```

Output:

Case 1:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex q8.l && gcc lex.yy.c && ./a.out

File not found.
```

Case 2:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex q8.l && gcc lex.yy.c && ./a.out

Length of longest word: 8

Longest word: compiler
```

10. Write a LeX program to copy a C program replacing each instance of the keyword float by double.

Source Code:

```
%{
    #include <stdio.h>

    FILE *fp;
%}

%%
float   {fputs("double", fp);}
.|\n    {fputs(yytext, fp);}
<<EOF>> {return 0;}

%%

int main(void)
{
    char filename1[100];
    char filename2[100];

    printf("Enter the source C program file : ");
    fgets(filename1, sizeof(filename1), stdin);

    printf("Enter the destination C program file : ");
    fgets(filename2, sizeof(filename2), stdin);

    filename1[strcspn(filename1, "\n")] = '\0';
    filename2[strcspn(filename2, "\n")] = '\0';

    yyin = fopen(filename1, "r");
    fp = fopen(filename2, "w");

    yylex();
    fclose(yyin);
    fclose(fp);

    printf("Your work is done. Check the destination file for the output.\n");
```

```
    return 0;
}

int yywrap()
{
    return 1;
}
```

Output:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex 10.l && gcc
lex.yy.c && ./a.out
Enter the source C program file : text.c
Enter the destination C program file : copy.c
Your work is done. Check the destination file for the output.
```

Text.c:

```
#include <stdio.h>
int main() {
    float a = 10.5;
    float b = 20.3;
    float sum;
    sum = a + b;
    printf("Sum = %f", sum);
    return 0;
}
```

Copy.c

```
#include <stdio.h>
int main() {
    double a = 10.5;
    double b = 20.3;
    double sum;
    sum = a + b;
    printf("Sum = %f", sum);
    return 0;
}
```

11. Write a LeX program to count total number of tokens that converts a file (i.e. sequence of words seperated by white space) through the following rules when u encounter a word :-

a. If the first letter is a consonant move it to the end of the word and then add y.

Source Code:

```
%{
#include<stdio.h>
#include<string.h>
#include<ctype.h>
int count = 0;

int isVowel(char c) {
    c = tolower(c);
    return (c=='a'||c=='e'||c=='i'||c=='o'||c=='u');
}
%}

%%
[a-zA-Z]+   {
                count++;

                if(isVowel(yytext[0])) {
                    printf("%s ", yytext);
                } else {
                    int len = strlen(yytext);

                    for(int i=1; i<len; i++)
                        printf("%c", yytext[i]);

                    printf("%cay ", yytext[0]);
                }
            }

[ \t\n]+    ;
%%

int yywrap() {
    return 1;
```

```
}

int main() {
    printf("Enter a sentence:\n");
    yylex();
    printf("\nTotal tokens = %d\n", count);
    return 0;
}
```

Output:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex q12.l && gcc lex.yy.c && ./a.out
Enter a sentence:
cat dog apple boy sky
atcay ogday apple oybay kysay
```

b. **If the first letter is a vowel , just add ly to the end of the word . All non letters are copied intact to the output.**

Source Code:

```
%{
#include<stdio.h>
#include<string.h>
#include<ctype.h>

int count = 0;

int isVowel(char c) {
    c = tolower(c);
    return (c=='a'||c=='e'||c=='i'||c=='o'||c=='u');
}
%}

%%
[a-zA-Z]+   {
                count++;

                if(isVowel(yytext[0])) {
                    printf("%sly", yytext);
                } else {
                    printf("%s", yytext);
                }
            }

[^a-zA-Z]+  {
                printf("%s", yytext);   // print non-letters as it is
            }
%%

int yywrap() {
    return 1;
}

int main() {
    printf("Enter a sentence:\n");
    yylex();
    printf("\nTotal tokens = %d\n", count);
```

```
    return 0;
}
```

Output:

```
sukdeb@SUKDEB:~/sukdeb/Compile Design practical/lex$ flex q12.l && gcc
lex.yy.c && ./a.out
Enter a sentence:
apple is good, dog runs fast! orange and ink are useful.
apple is oodgay ,ogday unsray astfay !orange and ink are useful .
```